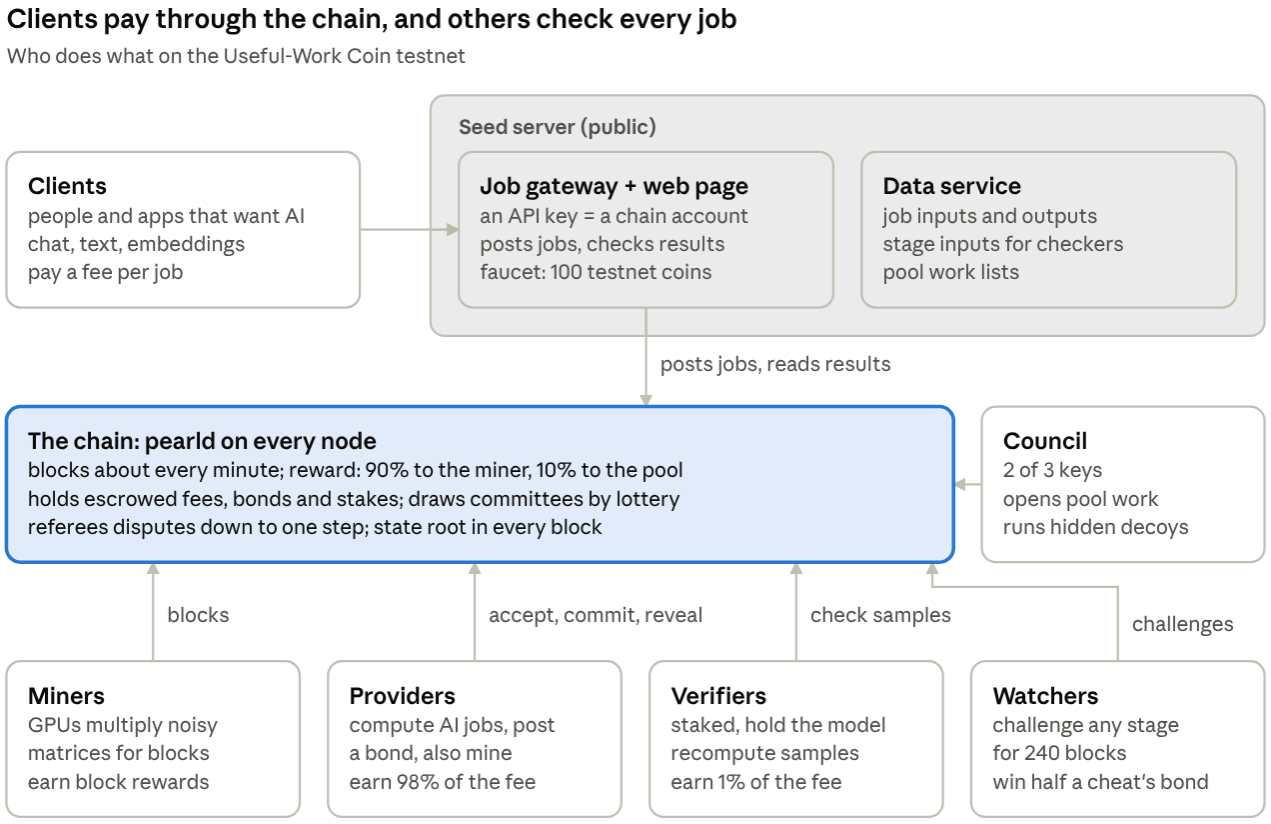
Useful-Work Coin: System Design Guide

Nov 11, 2026 · @Sid

Useful-Work Coin is a GPU-mined coin whose proof of work is matrix multiplication, joined to an on-chain market where clients pay providers for AI jobs that staked verifiers and bonded watchers check, and cheaters lose their deposits. This guide explains how the parts fit together, as the testnet runs today (Rules 5).

At a glance



system overview · 9 parts and what each sends the chain

Every payment, bond and check goes through the chain; the seed server only stores job data and offers the API and web page, so it can't change who is paid.

The chain and mining

The chain is a fork of Pearl, a Bitcoin-style node (Go, derived from btcd) whose proof of work is matrix multiplication. Useful-Work Coin adds a job module on top: consensus rules that run the AI job market inside every block.

Blocks. A block is a header plus transactions, as in Bitcoin. The header commits to the previous block, the time and one Merkle root over the transactions. Blocks come about once a minute on the testnet. A block may hold no transactions besides its coinbase; blocks keep coming whether or not anyone posts jobs.

Proof of work. A miner multiplies large matrices with random noise mixed in. The noise is derived from the block header, so a result counts only for that block and cannot be precomputed or reused. Small tiles of each product are hashed; a tile whose hash falls below the target wins the block, and the block carries a zero-knowledge proof that the winning tile came from a correctly computed product. Every node verifies that proof. Only multiplications with an inner dimension of at least 2,048 can mine, which is the size of real model layers.

Block reward. Supply is capped at 2.1 billion coins and the subsidy decays smoothly over a four-year constant: about 999 coins per block on the testnet today.

- **90%** goes to the block's miner, spendable after 20 more blocks.
- **10%** goes to the pool account, which pays for public AI work (see Pool work).
- Transaction fees go to the miner, as in Bitcoin.

Job state in the block. Job operations ride in ordinary transactions as `OP_RETURN` outputs. Coins enter the job module by deposit (an unspendable output credited to an account) and leave by withdrawal (paid by a later coinbase). After each block, every node applies the job rules and commits the resulting state root in the coinbase, so all nodes must agree on every balance, bond and job. A job operation that fails its checks changes nothing and still pays its fee, so a block is never invalid because of a job.

Useful work: AI jobs that also mine

A model's layers are mostly large matrix multiplications, the same operation mining uses. A provider started with `--mine` feeds every large multiplication of an AI job through the mining kernel, with the block's noise added and then removed exactly. The job gets its correct result, and the same work buys lottery tickets for the current block.

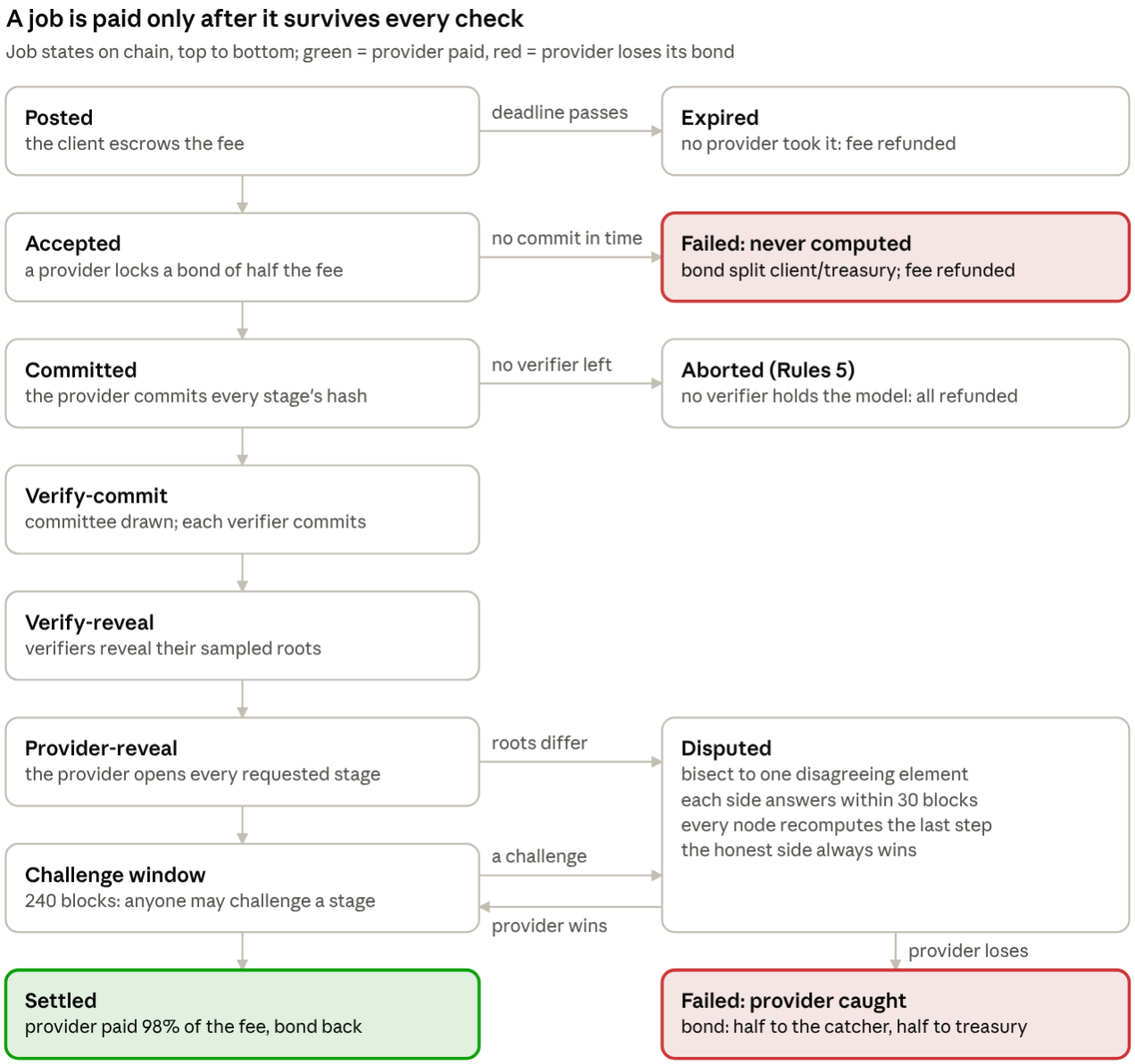
λ (lambda) is the share of plain-mining speed a GPU keeps while doing AI work this way. Measured live on the testnet, with jobs batched into GPU passes: about 0.37 on an RTX 3080 and 0.34 on an RTX 5090. A provider therefore earns its job fees and still keeps about a third of its mining chances.

Why results must be bit-identical. Verification compares hashes of every intermediate result. If two honest GPUs produced even one different bit, an honest provider would lose a dispute. So models run on a deterministic runtime, not on ordinary floating-point kernels:

- every float operation is fixed to one IEEE fp32 result (the reference computes each op in float64 and rounds once);
- integer and fixed-point sums are exact;
- weights and activations are quantized to 7 bits after a Hadamard rotation (W7A7), with no measurable loss in perplexity for Qwen3;
- attention uses int8 with 14-bit probabilities, and rotary tables are pinned.

The reference runtime (`detrt`) is slow but simple; the fast GPU engine (`detgpu`) is checked to match it bit for bit. Results are identical on RTX 3080, 3080 Ti, 5090 and CPU, across Qwen3, Llama and Mistral, including layers split over two GPUs.

A job's life



job lifecycle · 8 states, 5 ways to end early

A job that loses a dispute is refunded to its client, and the gateway posts it again. A challenger that loses pays half its bond to the provider; the job then returns to its challenge window and settles normally. Pool units follow the same path from Accepted, with the pool as their client.

Verification: committees, watchers and disputes

Nobody re-runs a whole job. A job is cut into **stages** (each sequence through one group of layers, plus a head stage per sequence for text generation), the provider commits a hash of every stage's result, and checkers recompute random stages. Any disagreement is narrowed down until the chain itself can recompute one tiny step.

1. Commitment. In `TRACE_COMMIT` the provider posts a salted Merkle root over all stage roots and the root of the outputs it delivered. Salts keep the unopened stage roots hidden, so nobody can copy them.

2. Committee. Two blocks later the chain draws k verifiers by a stake-weighted lottery seeded with that block's hash. From Rules 5 only staked verifiers that declared the job's model can be drawn; the client and provider never can. Each verifier picks n stage positions from its own secret salt, recomputes them, and commits a hash of its answers (`VERIFY_COMMIT`), then reveals them (`VERIFY_REVEAL`). Committing first stops verifiers from copying each other.

3. Provider reveal. The provider opens every requested stage with a Merkle proof (`TRACE_REVEAL`). A requested stage left unopened slashes it. A verifier root that differs from the provider's opens a dispute.

4. Challenge window. For 240 blocks (about 4 to 5 hours) anyone may `CHALLENGE` any stage with their own root and a bond. Watchers do this as sampled checks. If the provider's opened root equals the watcher's, it was just a check: the watcher pays a small watch fee to the provider and gets the rest of its bond back. If they differ, a dispute game starts.

5. Dispute game. The provider and challenger bisect: first between the stage's body and its output, then down the Merkle trees of the stage's rows, until they disagree on one element of one operation. Each move must come within 30 blocks or that side loses. A party that needs a stage's input asks for it on chain (`DATA_REQUEST`), and the provider must serve it (`DATA_SERVE`). The last step is `ONE_STEP`: every full node recomputes that single element with a Go referee that matches the Python runtime bit for bit. The honest side always wins.

6. Outcome. A provider that loses forfeits its bond: half to whoever caught it, half to the treasury, and the client's fee comes back. A challenger that loses forfeits its bond: half to the provider, half to the treasury. With no dispute, the job settles at the end of the

window.

Money: who pays and who is paid

A client's fee is held in escrow from the moment the job is posted; an honest provider ends up with 98% of it plus its bond back, and every penalty goes half to whoever caught the fault and half to the treasury. Example from the testnet run of 2026-11-11: a 5-coin job paid the provider 4.9 coins, the verifier 0.05 and the treasury 0.05, and returned the provider's 2.5-coin bond.

Money	Paid by	Normal outcome	When something goes wrong
Block reward (about 999 coins a block now)	new issuance	90% to the miner, 10% to the pool account	–
Transaction fee	the sender	the block's miner	–
Job fee F	client, escrowed at post	provider gets F minus verifier pay and treasury share	not accepted by the deadline: refunded; provider caught: refunded
Verifier pay V = max(1% of F, k × n × 0.00001)	out of F	shared by the verifiers that revealed	unpaid share goes to the treasury
Treasury share T = 1% of F	out of F	treasury	–
Provider bond B ≥ 50% of F	provider, locked at accept	returned at settlement	caught: ½ to the catcher, ½ to the treasury; accepted but never committed: ½ to the client, ½ to the treasury
Verifier stake (at least 0.01 coin)	verifier	kept while staked; unstaking takes 1,440 blocks	0.0005 coin to the treasury each time a drawn verifier doesn't commit or reveal

Money	Paid by	Normal outcome	When something goes wrong
Challenge bond (≥ 10% of B, at least 0.001 coin)	watcher or challenger	a check that agrees costs 0.0001 coin, paid to the provider; the rest returns	wrong challenge: ½ to the provider, ½ to the treasury
Pool unit payout	pool category budget	per epoch (60 blocks): budget ÷ units delivered, less 1% to the treasury	failed or abandoned unit: released for another claim, its verifier pay returns to the budget
Decoy jackpot (0.05 coin)	a forced-error reserve, taken from the treasury	split between whoever caught the decoy	missed decoys are counted on chain
Model registration bond (0.01 coin)	whoever registers a model	locked with the model	–

Amounts are in coins (1 coin = 10^8 base units). Every coin is accounted for at every height: coins outside the job module plus balances inside it always equal total issuance.

Pool work: public jobs paid from the block reward

The 10% of every block reward that goes to the pool account pays for standard AI work when there aren't enough paying clients, so providers stay busy and the network does useful work from day one.

The council. A fixed set of keyholders (3 on the testnet, any 2 of them needed) approves pool categories and forced-error commitments. One member sends `PROPOSE`, others `APPROVE`; with enough approvals the action runs as the council account, which no single key controls. A proposal that isn't approved within 1,440 blocks expires.

Categories. A category is a fixed public work list for one model: the Merkle root of its units (each unit's input), the number of units, each unit's shape (stages, sequence length, k and n), a deadline per claim, a share of pool income (cap) and a starting grant. On the testnet the units are WikiText passages, 8 sequences of 128 tokens each. Each block's pool income is split between the open categories by their caps; what is left stays in the pool reserve, which funds new categories (about 117,600 coins on 2026-11-11).

Claiming units. Any account can `POOL_CLAIM` a free unit with a Merkle proof that it is on the list and a bond. The claim becomes an ordinary job, already accepted, whose client is the pool, and it is verified exactly like a client job. Each unit is delivered once; a failed or

abandoned unit is released for someone else. Providers fetch work lists from the data service and check them against the on-chain root before using them.

Pricing. Nobody sets a price. Every 60 blocks (an epoch) the category's whole budget is split between the units delivered in that epoch. Fewer providers means more per unit. The latest payout becomes the unit price, which sets the bond for new claims. Nobody can post a pool job, so there is no way to pay yourself.

Canary units. A curator can hide a few units whose correct answers it already knows, committed with the category among dummy entries so nobody can tell which they are. After a provider commits a canary unit, the curator reveals it, and every stage of that job is checked. A provider that substituted a cheaper model fails every stage and is caught with certainty. A curator with a wrong answer loses the dispute and its own bond, so it can't harm an honest provider.

Forced errors and decoys: keeping checkers alert

If nearly every provider is honest, checking would almost never pay, and checkers would stop. So the council secretly runs **decoy** jobs with one deliberately wrong stage, and whoever catches one gets a jackpot.

1. **Commit.** The keyholders pick fresh decoy provider accounts and a secret stage key, and commit a hash of that record on chain (`FE_COMMIT`) with a reserve taken from the treasury. The secret is split between the keyholders (Shamir, any 2 of 3), so no single one holds it.
2. **Run.** From fresh client accounts they post ordinary jobs directed to the decoy providers. A decoy commits honest results except one wrong stage, chosen from the job id and the stage key, and defends every honest stage in disputes. The chain, verifiers and watchers can't tell a decoy from a client job.
3. **Catch.** A verifier that samples the wrong stage, or a watcher that checks it, wins the dispute as usual and takes half the decoy's bond.
4. **Reveal.** After every decoy job has ended, any 2 keyholders combine their shares and publish the record (`FE_REVEAL`). Each caught decoy pays its catchers the jackpot; each missed one is counted on chain. A commitment not revealed within 20,160 blocks (two weeks) expires and its reserve returns.

The counts measure how often the network catches a cheater. In the 2026-11-11 run, a verifier caught the first decoy on its wrong stage; the second settled unnoticed, because its committee sampled other stages and no watcher checked the wrong one. Run 1 missed one of three. A weakness the design accepts: the keyholders who run decoys know which jobs they are.

Models and jobs

A model is used on chain only after `MODEL_REG` registers exactly what it is: a Merkle root over every weight row, a hash of the deterministic runtime's source code, its shape (layers, width, context, architecture) and its number of layer groups. The model id is the hash of that record, so a job names its exact model and runtime. The weights root lets a dispute's last step prove the one weight row it uses.

Two kinds of job.

- **Embedding jobs:** sequences in, one vector per sequence out. A job has batches × G stages, where G is the model's layer groups.
- **Generation jobs** (from Rules 3): prompts in, generated text out. Each sequence gets one more stage, the head, which scores the vocabulary and picks each next token. Sampling is deterministic: Gumbel-max noise from a counter-based hash seeded with the job id, temperature up to 4, or greedy at 0. A finished generation is re-checked in one pass, because feeding the whole text back gives bit for bit the same states as generating it.

Registered models on the testnet (Llama and Mistral need Rules 4's architecture flag: no per-head query/key norms):

Model	Model id	Use	Layer groups	Width	Checked by
Qwen3 1.7B	bf027e9a…	chat, embeddings	7	2,048	box 71 (2 verifiers)
Llama 3.2 3B Instruct	a0d88293…	chat	7	3,072	.245 (1 verifier)
Mistral 7B Instruct v0.2	b7091e78…	chat	8	4,096	.245 (1 verifier)
Llama 3.1 8B (base)	aaa6c808…	text completion	8	4,096	.245 (1 verifier)

All four allow 1,024 tokens of context. Weights are the published Hugging Face checkpoints; each node builds a weights store from them and checks it against the registered root.

Rule versions

The job rules change only at a scheduled block height, so every node switches at the same block and a new node can still sync the whole chain from genesis. Each new version ships in a node binary that every node must run before that height; before scheduling, the new binary replays the entire chain and must reach the same state root as the live nodes.

Rules	From block	What it added
5	4,280 (2026-11-11)	committees only from verifiers that declared the job's model (`VERIFIER_MODELS`); no claim or accept for a model nobody can verify; a job whose verifiers all left is cancelled and refunded
4	2,700 (2026-11-10)	Llama and Mistral architectures
3	2,120 (2026-11-08)	generation jobs: head stages, generation models, verified text output
2	1,400 (2026-11-07)	jackpots only for a catch on the decoy's wrong stage; jobs directed to one provider; challenge bonds of 10% of the provider's bond
1	genesis (2026-11-06)	the job module: jobs, committees, challenges, dispute games, pool categories, canaries, watchers, forced errors

An old node keeps working until the next activation height, then stops following the chain. That is why the README always links the current miner package.

Software and where it runs

The system is a handful of programs: one node, one mining stack, one agent program with several roles, and two services on the seed server.

Program	What it does	Runs on
`pearld` (Go)	full node: blocks, mining proofs, the job rules and state root, mempool, RPC	box 71, .245, the seed server, every miner package
`uwcctl`, `prlctl`	command-line tools: keys, payments, deposits, job operations; node RPC	next to each node
Mining gateway (Python)	gets block templates from the node, hands work to the GPU miners, submits winning proofs	next to each mining node

Program	What it does	Runs on
Noise miner `uwc_ampere` (Python + CUDA)	per GPU: noise generation, the noisy matrix multiply, tile hashing	box 71 GPUs 0 and 2, the 5090
Agents `agent.roles` (Python + CUDA)	one program, six roles: provider, verifier, watcher, client, decoy, curator; all GPU work on the deterministic runtime	box 71, .245, the 5090
Data service `agent.data`	stores job inputs, outputs, stage inputs and pool work lists; anyone reads, only signed uploads from the right account are accepted	seed server
Job gateway `agent.gateway`	the jobs web page and an OpenAI-compatible API; posts and follows jobs for API keys	seed server, behind HTTPS
`uwc-miner` package	one folder with Python, CUDA libraries, the node, mining gateway, miner and a Qwen3 provider; `./uwc-miner --jobs` runs it all	anyone's Linux machine with an NVIDIA GPU

The testnet's machines

Machine	GPUs	Role
Box 71	3 × RTX 3080	node, mining, Qwen3 provider, 2 verifiers, 2 watchers, decoy runner, council keys
.245	3 × RTX 3080, 2 × 3080 Ti	node, Llama 8B provider, the verifier and watcher for Llama and Mistral; package builds and uploads
5090 box	RTX 5090	the public miner package (mining and Qwen3 jobs), Llama 3B and Mistral providers
Seed server (165.232.182.224)	none	public seed node, data service, job gateway and web page

The miner package (2.9 GB) is downloaded from public storage and needs Linux x86-64 (or WSL2) with NVIDIA driver 580 or newer. It installs nothing on the system and keeps its data in `~/.uwc-miner/`.

The gateway and jobs page

The job gateway lets people use the network like an ordinary AI API: it holds a chain account per API key, turns each request into jobs on chain, checks the delivered results against the provider's commitment, and follows each job to settlement. The jobs page at 165-232-182-224.sslip.io is a front end to that API.

- **Accounts.** Creating a key needs a signup code (at most 3 keys per address per day). The faucet puts 100 coins into the new key's jobs balance and 2 coins into its wallet for transaction fees.
- **Requests.** Chat (Qwen3, Llama 3B, Mistral), text completion (all four, including the Llama 8B base model), embeddings (Qwen3) and batches of up to 256 inputs from a file, with results as a JSON-lines download. Chat streams the provider's partial text while it generates.
- **Pricing.** A fee per 1,000 tokens, set per model (1.5 coins for Qwen3, 2.5 for Llama 3B, 5 for Mistral, 6 for Llama 8B), at least 0.5 coins a job. A batch's generation prompts go 16 to a job.
- **Retries.** A job nobody takes is posted again with a 50% higher fee; a job whose provider lost a dispute is posted again at the same fee; after 3 attempts the request fails.
- **API.** OpenAI-compatible: `/v1/chat/completions`, `/v1/completions`, `/v1/embeddings`, plus `/v1/batches`, `/v1/jobs` and `/v1/jobs/<id>/results`.

Trust levels tell the user how far along the checking is:

Trust	Meaning	Typical time
committed	the delivered results match what the provider committed on chain	a few minutes
verified	the verifier committee checked its sampled stages and agreed	about 10 minutes more
final	the challenge window passed with no successful challenge; the provider is paid	about 5 hours
disputed	a dispute game is in progress	until it ends
failed	the provider lost a dispute; the job is posted again	–

Job inputs are stored on the public data service so that anyone can check the work: the page and the tester note warn users to send only text that may be published.

Key parameters (testnet)

These are the values the testnet runs with under Rules 5; amounts in coins, times in blocks of about one minute.

Parameter	Value	What it controls
Supply cap	2.1 billion	all coins ever issued
Block subsidy	about 999 now, decaying	new coins per block
Coinbase maturity	20 blocks	when mined coins can be spent
Pool share	10% of the subsidy	funding for public AI work
Verifier share, treasury share	1% and 1% of each fee	verifier pay and the treasury's cut
Provider bond	at least 50% of the fee	what a cheating provider loses
Committee size k, samples each n	at least 2 and 2 (up to 16 and 64)	how many verifiers and stages per verifier
Large jobs	a fee of 5 or more block rewards needs k ≥ 3	extra checking for valuable jobs
Committee delay	2 blocks after commit	when the committee is drawn

Parameter	Value	What it controls
Verify commit, verify reveal, provider reveal	30 blocks each	reply windows
Challenge window	240 blocks	how long anyone may challenge
Dispute move, input serve windows	30 blocks each	how fast a party must answer
Minimum verifier stake	0.01	to be drawn for committees
No-show penalty	0.0005	a drawn verifier that doesn't answer
Verifier unbonding	1,440 blocks	stake stays slashable this long after unstaking
Models per verifier declaration	up to 16	Rules 5
Challenge bond	10% of the provider's bond, at least 0.001	what a wrong challenger loses
Watch fee	0.0001	cost of a check that agrees, paid to the provider
Jackpot	0.05 per caught decoy	reward for catching a forced error
Pool epoch	60 blocks	how often pool units are paid
Council	2 of 3 keys	approves categories and forced errors
Proposal window	1,440 blocks	council proposals expire after this
Forced-error reveal window	20,160 blocks	an unrevealed commitment expires after this
Withdrawal delay	6 blocks	from the job module back to the wallet
Model registration bond	0.01	locked per registered model

Known limits and open questions

The testnet works end to end, but these are not settled yet:

- **Catch rate.** Committees and watchers check samples, so a provider that corrupts one stage of a job is sometimes missed: one of two decoys in the 2026-11-11 run, one of three in run 1. The rate decides whether cheating pays, and should be measured before coins have value. Levers: more samples per verifier, more watcher checks, larger jackpots.
- **Verifier coverage.** Each non-Qwen model has one verifier, on one machine. If it goes away, Rules 5 refuses new work for that model rather than letting it through unchecked, but the model stops serving.
- **One seed node.** New miners find the network only through the seed server, which also runs the data service and the gateway.
- **A central council.** The same people hold the council keys and run the decoys. The design accepts that decoy runners know their decoys.
- **Little paid work.** The pool categories are used up; outside miners with `--jobs` mostly mine until new categories open or clients arrive.
- **Platforms.** The miner package is Linux only (Windows through WSL2); a native Windows build is planned.
- **Bugs.** Every testnet run so far has found a few, lately in the agent programs rather than the chain rules. The plan is a 24 to 48 hour soak test with outside testers and deliberate faults, and wider release after two clean runs in a row.

Sources

This guide draws on the project's own documents and code (repository `useful-work-coin`):

- *Useful-Work Coin: Design v3* (`spec/design-v3.pdf`): the original design and its economics
- `spec/job-module.md`: money flows and the job lifecycle
- `spec/stage-program.md`: stages, commitments and the dispute game
- `spec/pool-and-watchers.md`: the council, pool categories, canaries, watchers and forced errors
- `spec/generation.md`: generation jobs
- `spec/rules4-llama-mistral.md` and `spec/rules5-model-committees.md`: the latest rule versions
- `spec/step6-results.md` and `spec/step7-lambda.md`: the economic simulation and measured λ

- `node/jobstate/types.go` and `node/chaincfg/uwc_testnet.go` (Pearl fork): parameter values
- AICoin README and tester note: installing and testing